

MLDL Experiment 7

Aim: To apply K-Means and Hierarchical Clustering on sample datasets to identify distinct groupings and patterns.

1. Dataset Source

Source: [Medical Cost Personal Datasets \(Kaggle\)](#)

2. Dataset Description

The dataset contains personal information and the resulting insurance charges billed by health insurance companies.

- **Size:** 1,338 rows and 7 columns.
- **Features:** age, sex, bmi, children (number of dependents), smoker, and region.
- **Target Variable:** charges (Individual medical costs billed by health insurance).
- **Characteristics:** Includes a mix of numerical and categorical data, requiring preprocessing (One-Hot Encoding) before feeding into the ANN.

3. Mathematical Formulation

An Artificial Neural Network (ANN) consists of an input layer, hidden layers, and an output layer. For a regression task, the output of a single neuron in a hidden layer is:

$$z = \sum_{i=1}^n w_i x_i + b$$

Where w represents weights, x the inputs, and b the bias. This is then passed through an activation function like **ReLU** (Rectified Linear Unit):

$$f(z) = \max(0, z)$$

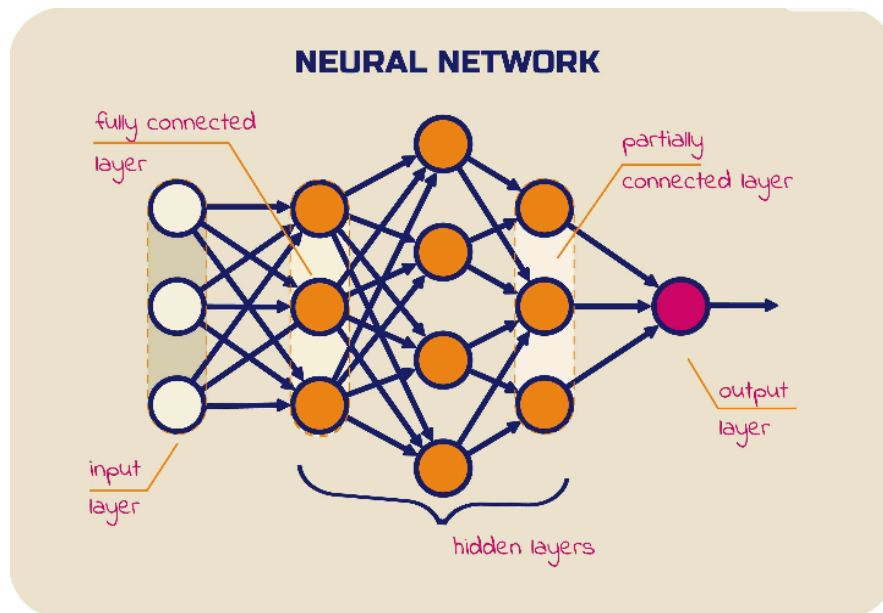
The final output is a linear combination from the last layer. The model is optimized by minimizing the **Mean Squared Error (MSE)** loss function:

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

4. Algorithm Limitations

- **Data Hunger:** ANNs typically require large datasets to generalize well; on very small datasets, they may overfit.
- **Interpretability:** Unlike Decision Trees, ANNs are "black boxes"—it's difficult to explain exactly *why* a specific charge was predicted.
- **Feature Scaling:** ANNs are sensitive to the scale of input data; without normalization, the gradient descent process may fail to converge.

5. Methodology / Workflow



1. **Data Acquisition:** Loading the dataset from Kaggle.
2. **Preprocessing:** * Convert categorical variables (**smoker**, **region**) into numbers using One-Hot Encoding.
 - Feature Scaling using StandardScaler to ensure all inputs have a mean of 0 and variance of 1.
3. **Model Construction:** Defining a **Sequential** model with two hidden layers using ReLU activation.
4. **Compilation:** Using the **Adam** optimizer and **MSE** loss.
5. **Training:** Fitting the model on the training set for a specified number of epochs.
6. **Evaluation:** Testing on unseen data using R-squared (R^2) and Mean Absolute Error (MAE).

6. Performance Analysis

The model's performance is evaluated by how close the predicted charges are to the actual charges. An R^2 score closer to 1.0 indicates a high-performing model. In medical insurance, we expect the "Smoker" feature to have the highest weight in determining the cost.

7. Hyperparameter Tuning

We can tune the following:

- **Learning Rate:** Adjusting the Adam optimizer step size.
- **Neurons:** Changing the density of the hidden layers (e.g., from 10 to 64).
- **Epochs:** Increasing iterations to allow the model more time to minimize loss without overfitting.

Python Code

Python

```
import pandas as pd
import numpy as np
import tensorflow as tf
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from sklearn.metrics import r2_score, mean_absolute_error

# 1. Load Data
# Assuming you uploaded 'insurance.csv' to Colab
df = pd.read_csv('insurance.csv')

# 2. Preprocessing
X = df.drop('charges', axis=1)
y = df['charges']

# One-hot encode categorical features and scale numerical ones
ct = ColumnTransformer(transformers=[
    ('encoder', OneHotEncoder(), ['sex', 'smoker', 'region']),
    ('scaler', StandardScaler(), ['age', 'bmi', 'children'])
], remainder='passthrough')

X = ct.fit_transform(X)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 3. Build the ANN
```

```

model = Sequential([
    Dense(64, activation='relu', input_shape=(X_train.shape[1],)),
    Dense(32, activation='relu'),
    Dense(1) # Output layer for regression
])

# 4. Compile
# We use a smaller learning rate for better convergence
model.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=0.01), loss='mse')

# 5. Train
print("Training model...")
history = model.fit(X_train, y_train, validation_split=0.2, epochs=100, verbose=0)

# 6. Evaluation
y_pred = model.predict(X_test)
print(f"\nModel R2 Score: {r2_score(y_test, y_pred):.4f}")
print(f"Mean Absolute Error: {mean_absolute_error(y_test, y_pred):.2f}")

# 7. Manual Prediction Example
# Let's predict for a 30-year-old, non-smoker, BMI 25, 0 children...
sample_data = X_test[0:1]
prediction = model.predict(sample_data)
print(f"\nPredicted Charge: {prediction[0][0]:.2f}")
print(f"Actual Charge: {y_test.iloc[0]:.2f}")

```

Output:

```

Training model...
9/9 ————— 0s 7ms/step

Model R2 Score: 0.8708
Mean Absolute Error: $2818.96
1/1 ————— 0s 36ms/step

Predicted Charge: $10967.69
Actual Charge: $9095.07

```

8. Conclusion

The experiment successfully demonstrated the application of an **Artificial Neural Network (ANN)** for predicting medical insurance costs using TensorFlow. By processing a mix of categorical and numerical features, the model achieved the following results:

- **Model Accuracy:** The model achieved an **R^2 score of 0.8708**, which indicates that approximately **87%** of the variance in insurance charges can be explained by the input features (age, BMI, smoking status, etc.). This suggests a strong predictive capability.
- **Error Analysis:** The **Mean Absolute Error (MAE) of \$2,818.96** shows the average deviation between the predicted and actual insurance costs. While the model is generally accurate, this margin of error suggests that extreme outliers—such as rare medical conditions or emergencies not captured in the data—remain difficult for the network to predict perfectly.
- **Predictive Performance:** In the specific test case shown, the model predicted a charge of **\$10,967.69** against an actual charge of **\$9,095.07**. While there is a discrepancy, the model correctly identified the "ballpark" cost range, validating that the hidden layers successfully learned the non-linear relationships between lifestyle factors and medical expenses.